# Estudo Dirigido — FIPS-197: Advanced Encryption Standard (AES)

# ADS - Padrão mundial de criptografia usado para proteger dados confidenciais.

*Por Raissa Genesio da Silva*

## 1. Visão geral do AES

O AES — *Advanced Encryption Standard* — é um algoritmo de criptografia simétrica de blocos. Isso significa que ele utiliza uma chave secreta para transformar dados legíveis (*plaintext*) em dados cifrados (*ciphertext*), e a mesma chave é usada no processo inverso de decifragem. O AES trabalha sempre com blocos de **128 bits**, mas pode utilizar chaves de **128, 192 ou 256 bits**.

As três configurações definidas pelo padrão são:

| AES | Chaves | Blocos | Rodadas |
|---|---|---|---|
| AES - 128 | 128 | 128 | 10 |
| AES - 192 | 192 | 128 | 12 |
| AES - 256 | 256 | 128 | 14 |

A quantidade de rodadas depende do tamanho da chave. nist.fips.197. Quanto maior a chave, mais rodadas são necessárias para garantir a margem de segurança contra métodos de criptoanálise

## 2. O que são as rodadas AES?

**As rodadas do AES são como repetições de um processo de embaralhamento de um baralho de cartas.**

**Funciona basicamente como:**

Um ciclo repetido: O algoritmo pega o bloco de dados e aplica a mesma sequência de 4 passos matemáticos (trocar bytes, deslocar linhas, misturar colunas e somar um pedaço da senha).

Camadas de proteção: Uma única rodada não é segura. Ao passar os dados por várias rodadas seguidas, o resultado final vira um ruído estatístico indecifrável.

O objetivo prático: Fazer com que a alteração de apenas 1 bit na entrada inicial mude cerca de 50% de todo o resultado final, impedindo que qualquer invasor encontre padrões lógicos para adivinhar a mensagem original.

### Estrutura do Processo:

O bloco de dados (sempre fixo em 128 bits, organizado em uma matriz $4 \times 4$ de bytes) passa por três fases:

1. **Rodada Inicial (Round 0):** Apenas uma operação preliminar de **AddRoundKey** (consiste na operação lógica XOR ($\oplus$) realizada posição por posição (byte a byte) entre a matriz de estado inicial e a matriz da chave. ).
2. **Rodadas Padrão (Rounds 1 a $N_r - 1$):** Executam o ciclo completo das 4 transformações:
    - **SubBytes** (não-linearidade / confusão)
    - **ShiftRows** (permutação / difusão entre colunas)
    - **MixColumns** (mistura linear / difusão dentro das colunas)
    - **AddRoundKey** (adição da subchave gerada para a rodada)
3. **Rodada Final (Round $N_r$):** Executa **SubBytes, ShiftRows e AddRoundKey, omitindo o MixColumns** (essa omissão simplifica e torna a estrutura de descriptografia simétrica e reversível em hardware/software).

## Explicação do processo de rodadas:

**Fase 0 - Inicial (Rodada 0)** AddRoundKey()**:**

- Ocorre antes do ciclo iterativo principal começar.
- Aplica-se apenas a transformação AddRoundKey() inicial. Nesta transformação, a matriz de estado original (os dados de entrada) é combinada com a primeira parte da chave gerada através de uma operação XOR.

  Exemplo:

  [Entrada : 32 43 f6 a8 88 5a 30 8d 31 31 98 a2 e0 37 07 34]

PASSO 1 - Organizar em state:

$$\begin{bmatrix} 32 & 88 & 31 & e0 \\ 43 & 5a & 31 & 37 \\ f6 & 30 & 98 & 07 \\ a8 & 8d & a2 & 34 \end{bmatrix}$$

[Chave inicial : 2b 7e 15 16 28 ae d2 a6 ab f7 15 88 09 cf 4f 3c]

PASSO 2 - Organizar como matriz:

$$\begin{bmatrix} 2b & 28 & ab & 09 \\ 7e & ae & f7 & cf \\ 15 & d2 & 15 & 4f \\ 16 & a6 & 88 & 3c \end{bmatrix}$$

PASSO 3: fazer XOR posição por posição

$$\begin{bmatrix} 32 & 88 & 31 & e0 \\ 43 & 5a & 31 & 37 \\ f6 & 30 & 98 & 07 \\ a8 & 8d & a2 & 34 \end{bmatrix} \oplus \begin{bmatrix} 2b & 28 & ab & 09 \\ 7e & ae & f7 & cf \\ 15 & d2 & 15 & 4f \\ 16 & a6 & 88 & 3c \end{bmatrix}$$

Resulta em:

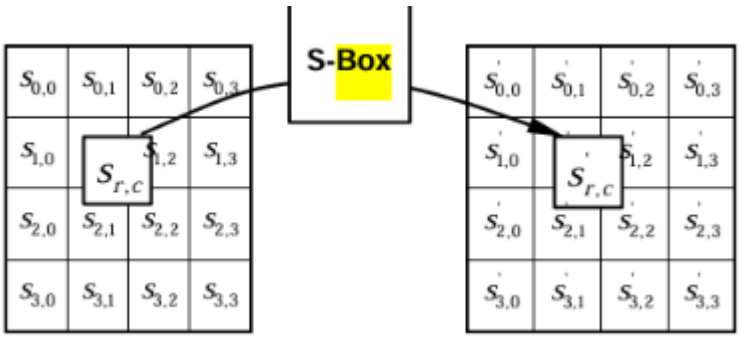
$$\begin{bmatrix} 19 & a0 & 9a & e9 \\ 3d & f4 & c6 & f8 \\ e3 & e2 & 8d & 48 \\ be & 2b & 2a & 08 \end{bmatrix}$$

**Fase 1 - Etapa SubBytes: :** É uma transformação de substituição de bytes não-linear. Cada byte da matriz de estado é substituído de forma completamente independente utilizando uma tabela de substituição conhecida como **S-box.**

**Tabela Sbox:**

| | | y | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | a | b | c | d | e | f |
| x | 0 | 63 | 7c | 77 | 7b | f2 | 6b | 6f | c5 | 30 | 01 | 67 | 2b | fe | d7 | ab | 76 |
| | 1 | ca | 82 | c9 | 7d | fa | 59 | 47 | f0 | ad | d4 | a2 | af | 9c | a4 | 72 | c0 |
| | 2 | b7 | fd | 93 | 26 | 36 | 3f | f7 | cc | 34 | a5 | e5 | f1 | 71 | d8 | 31 | 15 |
| | 3 | 04 | c7 | 23 | c3 | 18 | 96 | 05 | 9a | 07 | 12 | 80 | e2 | eb | 27 | b2 | 75 |
| | 4 | 09 | 83 | 2c | 1a | 1b | 6e | 5a | a0 | 52 | 3b | d6 | b3 | 29 | e3 | 2f | 84 |
| | 5 | 53 | d1 | 00 | ed | 20 | fc | b1 | 5b | 6a | cb | be | 39 | 4a | 4c | 58 | cf |
| | 6 | d0 | ef | aa | fb | 43 | 4d | 33 | 85 | 45 | f9 | 02 | 7f | 50 | 3c | 9f | a8 |
| | 7 | 51 | a3 | 40 | 8f | 92 | 9d | 38 | f5 | bc | b6 | da | 21 | 10 | ff | f3 | d2 |
| | 8 | cd | 0c | 13 | ec | 5f | 97 | 44 | 17 | c4 | a7 | 7e | 3d | 64 | 5d | 19 | 73 |
| | 9 | 60 | 81 | 4f | dc | 22 | 2a | 90 | 88 | 46 | ee | b8 | 14 | de | 5e | 0b | db |
| | a | e0 | 32 | 3a | 0a | 49 | 06 | 24 | 5c | c2 | d3 | ac | 62 | 91 | 95 | e4 | 79 |
| | b | e7 | c8 | 37 | 6d | 8d | d5 | 4e | a9 | 6c | 56 | f4 | ea | 65 | 7a | ae | 08 |
| | c | ba | 78 | 25 | 2e | 1c | a6 | b4 | c6 | e8 | dd | 74 | 1f | 4b | bd | 8b | 8a |
| | d | 70 | 3e | b5 | 66 | 48 | 03 | f6 | 0e | 61 | 35 | 57 | b9 | 86 | c1 | 1d | 9e |
| | e | e1 | f8 | 98 | 11 | 69 | d9 | 8e | 94 | 9b | 1e | 87 | e9 | ce | 55 | 28 | df |
| | f | 8c | a1 | 89 | 0d | bf | e6 | 42 | 68 | 41 | 99 | 2d | 0f | b0 | 54 | bb | 16 |

**Ilustração de como é feita a substituição dos bytes**




Por exemplo, para o byte 19:

linha 1 coluna 9, temos :

19 – d4

a0:

linha a, coluna 0:

a0 – e0

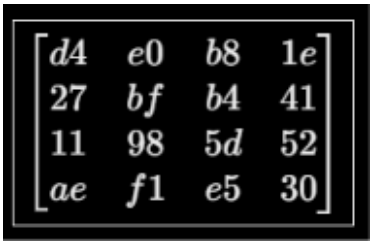
E assim por diante para todos os 16 bytes

Ficamos com:

$$\begin{bmatrix} 19 & a0 & 9a & e9 \\ 3d & f4 & c6 & f8 \\ e3 & e2 & 8d & 48 \\ be & 2b & 2a & 08 \end{bmatrix} \rightarrow \begin{bmatrix} d4 & e0 & b8 & 1e \\ 27 & bf & b4 & 41 \\ 11 & 98 & 5d & 52 \\ ae & f1 & e5 & 30 \end{bmatrix}$$

**Fase 2 - ShiftRows:** Consiste num deslocamento cíclico dos bytes. As últimas três linhas da matriz de estado são deslocadas lateralmente em diferentes números de posições. A primeira linha da matriz não sofre qualquer deslocamento.

Partindo do State que obtivemos após o SubBytes



$$\begin{bmatrix} d4 & e0 & b8 & 1e \\ 27 & bf & b4 & 41 \\ 11 & 98 & 5d & 52 \\ ae & f1 & e5 & 30 \end{bmatrix}$$

Aplicamos:

- linha 0: desloca **0**
- linha 1: desloca **1 posição para a esquerda**
- linha 2: desloca **2 posições**
- linha 3: desloca **3 posições**

Então:

## Linha 0

Antes:

$$[d4\ e0\ b8\ 1e]$$

Não muda:

$$[d4\ e0\ b8\ 1e]$$

## Linha 1

Antes:

$$[27\ bf\ b4\ 41]$$

Desloca 1:

$$[bf\ b4\ 41\ 27]$$

**Linha 2**

Antes:

$$[11\ 98\ 5d\ 52]$$

Desloca 2:

$$[5d\ 52\ 11\ 98]$$

**Linha 3**

Antes:

$$[ae\ f1\ e5\ 30]$$

Desloca 3:

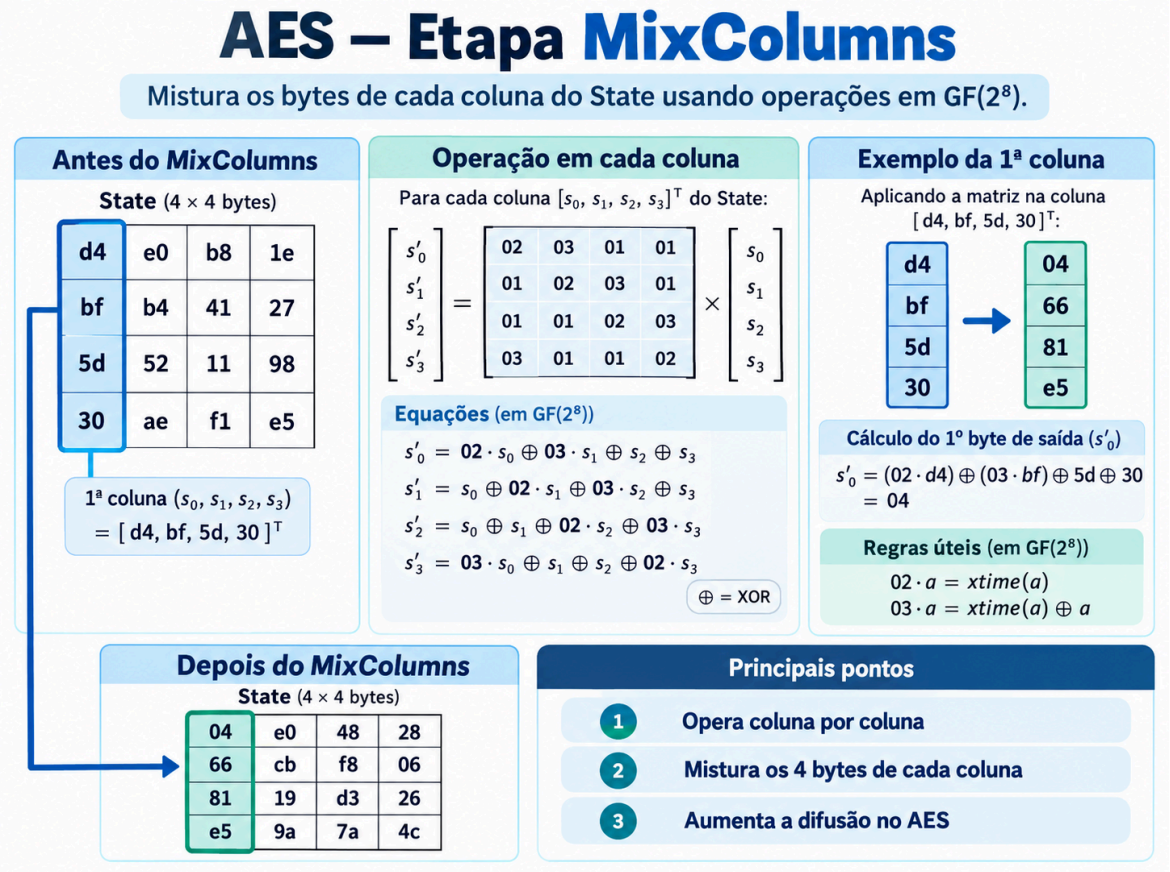
$$[30\ ae\ f1\ e5\ ]$$

O state final fica:

$$\begin{bmatrix} d4 & e0 & b8 & 1e \\ bf & b4 & 41 & 27 \\ 5d & 52 & 11 & 98 \\ 30 & ae & f1 & e5 \end{bmatrix}$$

A ideia do ShiftRows é **espalhar os bytes entre as colunas**. Isso prepara o State para o MixColumns

**Fase 3 - MixColumns():** essa transformação opera na matriz de estado coluna a coluna. Mistura os dados de cada coluna independentemente umas das outras. Matematicamente, as colunas são tratadas como polinómios e multiplicadas módulo $x^4 + 1$ por um polinómio fixo.




Após o **MixColumns teríamos:**

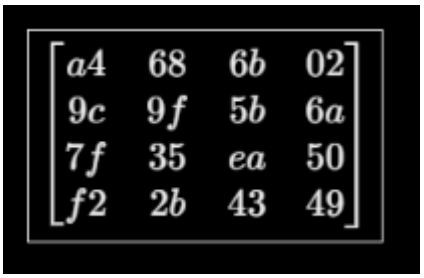
$$\begin{bmatrix} 04 & e0 & 48 & 28 \\ 66 & cb & f8 & 06 \\ 81 & 19 & d3 & 26 \\ e5 & 9a & 7a & 4c \end{bmatrix}$$

**Fase 4 -AddRoundKey():** Uma chave de rodada específica (derivada da chave principal através da rotina de expansão de chaves) é adicionada à matriz de estado utilizando uma operação lógica simples de bit a bit (XOR).

A Round Key da primeira rodada é:

$$\begin{bmatrix} a0 & 88 & 23 & 2a \\ fa & 54 & a3 & 6c \\ fe & 2c & 39 & 76 \\ 17 & b1 & 39 & 05 \end{bmatrix}$$

O State completo passa a ser:



E esse resultado vira a entrada da **próxima rodada**.

No contexto do nosso projeto, o objetivo é implementar esse algoritmo diretamente em hardware RTL.

O AES processa um bloco de:

$$128bits$$

Como:

$$1byte = 8bits$$

temos:

$$128/8 = 16bytes$$

Portanto, cada execução do AES recebe **16 bytes de dados**.

O AES não interpreta o significado desses bytes. Eles podem representar:

- texto;

- dados de sensores;
- arquivos;
- números;
- qualquer outra informação binária.

Para o AES, tudo é simplesmente uma sequência de bits.

Em resumo, a lógica:

$$K_0 = chave\ original$$

Depois a expansão gera:

$$K_1,\ K_2,\ K_3, \ldots, K_{10}$$

Então o fluxo fica:

$$Entrada \ \rightarrow \ AddRoundKey(K_0)$$ **(ETAPA 0)**

**Rodada 1:**

$$SubBytes \rightarrow ShiftRows \rightarrow MixColumns \rightarrow AddRoundKey(K_1)$$

**Rodada 2:**

$$SubBytes \rightarrow ShiftRows \rightarrow MixColumns \rightarrow AddRoundKey(K_2)$$

e assim por diante.

O AddRoundKey inicial ocorre com round = 0, antes da primeira aplicação da função de rodada. Depois, cada rodada usa a chave correspondente daquele round. nist.fips.197.